NAAVI

Technical Architecture Document

Version: 2.0 **Date:** March 19, 2026 **Status:** Phase 8 — Live & Fully Integrated **Prepared for:** CTO Review

1. What Naavi Is

Naavi is a life orchestration companion for active seniors. The primary user is Robert, 68, Ottawa. Robert's problem is that his tools — calendar, Gmail, Drive, health records, contacts, maps — do not talk to each other. Naavi connects them through a single conversational interface: he speaks or types, Naavi responds with voice and executes actions autonomously on his behalf — no app switching, no copy-pasting, no token management.

Zero-friction principle: Robert never authenticates, reconnects, manages tokens, or copies data between tools. Everything runs server-side automatically.

2. High-Level Architecture

```
+-----------------------------------------------------------------------+
|                         Robert's Device                               |
|                    (browser - any device)                             |
|                                                                       |
|  +-------------------------------------------------------------+  |
|  |              Naavi App  (Expo / React Native Web)              |  |
|  |                                                                |  |
|  |  Morning Brief    Conversation    Action Cards    Notes        |  |
|  |  (grouped,        (bubbles)       (Draft/Event/   (Memory +    |  |
```

Version: 2.0 **Date:** March 19, 2026 **Status:** Phase 8 — Live **Prepared for:** CTO Review

---

```
|  |  collapsible)                     Contact/Drive)  Drive tab)  |  |
|  |                                                               |  |
|  |  Voice Input                                                  |  |
|  |  +----------------+  +-------------------------------------+  |  |
|  |  | Whisper (mic)  |  | Web Speech API (green button)        |  |  |
|  |  | clip->Whisper  |  | real-time browser STT                |  |  |
|  |  +-------+--------+  +--------------+-----------------------+  |  |
|  |          +---------------------+                              |  |
|  |                                v                              |  |
|  |                    useOrchestrator.ts                         |  |
|  |                                v                              |  |
|  |               naavi-client.ts <-- buildSystemPrompt()         |  |
|  +----------------------------+-----------------------------------+  |
+------------------------------|-------------------------------------+
                               |
            +-------------------+-------------------+
            |                                       |
            v                                       v
+--------------------+                  +-------------------------+
|  Anthropic API     |                  |   Supabase              |
|  claude-sonnet-4-6 |                  |                         |
|  max_tokens: 2048  |                  |  Auth (Google OAuth)    |
|  Returns JSON:     |                  |  Postgres + pgvector    |
|  {speech,          |                  |  14 Edge Functions      |
|   actions[],       |                  |  Row-Level Security     |
|   pendingThreads}  |                  +----------+--------------+
+--------------------+                             |
                                  +----------------+------------------+
                                  |         Google APIs               |
                                  |  Gmail - Calendar - Drive         |
                                  |  Contacts - Maps Distance Matrix  |
                                  |  OpenAI Whisper + Embeddings      |
                                  +-----------------------------------+
```

3. Technology Stack

Layer	Technology	Version	Notes

NAAVI

Version: 2.0 **Date:** March 19, 2026 **Status:** Phase 8 — Live **Prepared for:** CTO Review

---

App framework	Expo / React Native	SDK 55	Single codebase: web, Android, iPhone
Language	TypeScript + React	React 19.2.4	Strict typing throughout
Navigation	Expo Router v5	~55.0.5	File-based routing
AI model	Claude Sonnet 4.6	claude-sonnet-4-6	Orchestration + fragment extraction
AI SDK	@anthropic-ai/sdk	^0.79.0	Used in naavi-chat Edge Function
Voice input (clip)	OpenAI Whisper	whisper-1	Via transcribe-memo Edge Function
Voice input (live)	Web Speech API	Browser native	Chrome/Android real-time STT

Voice output	Web Speech Synthesis	Browser native	Prefers Microsoft/Google neural voices
Vector embeddings	OpenAI text-embedding-3-small	1536 dimensions	Knowledge fragment semantic search
Vector storage	pgvector (Postgres extension)	—	Cosine similarity via RPC
Backend	Supabase	—	Auth, Postgres, Edge Functions (Deno)
Auth	Supabase Google OAuth	—	Captures Google refresh token at sign-in
Hosting	Vercel	—	Auto-deploys from GitHub push (~90s)
Source control	GitHub	—	github.com/munk2207/naavi-app

i18n	i18next + react-i18next	^23.0.0	English / Canadian French

4. Repository and Folder Structure

```
Naavi/
+-- mobile/                          <-- THE LIVE APP
|   +-- app/
|   |   +-- index.tsx                <-- Main screen (brief, conversation, cards)
|   |   +-- notes.tsx                <-- Notes repository (Memory + Drive tabs)
|   |   +-- settings.tsx             <-- Auth, language settings
|   |   +-- _layout.tsx              <-- Navigation shell
|   +-- hooks/
|   |   +-- useOrchestrator.ts       <-- Conversation loop + action executor
|   |   +-- useVoice.ts              <-- Web Speech API integration
|   |   +-- useWhisperMemo.ts        <-- Whisper clip recording + transcription
|   +-- lib/
|       +-- naavi-client.ts          <-- System prompt builder + Claude call
|       +-- supabase.ts              <-- Supabase client + DB helpers
|       +-- calendar.ts              <-- Google Calendar fetch + BriefItem mapping
|       +-- gmail.ts                 <-- Gmail fetch + send
|       +-- drive.ts                 <-- Drive search + save + email attachment
|       +-- contacts.ts              <-- 3-tier contact resolution
|       +-- memory.ts                <-- People table + person context injection
|       +-- knowledge.ts             <-- Knowledge fragment ingest + search
|       +-- maps.ts                  <-- Travel time fetch + leave-by calculation
|       +-- weather.ts               <-- Ottawa weather fetch
|
+-- supabase/
|   +-- config.toml                  <-- verify_jwt = false on all functions
|   +-- functions/
|       +-- naavi-chat/              <-- Claude API proxy (API key server-side)
|       +-- transcribe-memo/         <-- OpenAI Whisper transcription
|       +-- sync-google-calendar/    <-- Pulls events to calendar_events table
|       +-- sync-gmail/              <-- Pulls messages to gmail_messages table
|       +-- search-google-drive/     <-- Drive file search
|       +-- save-to-drive/           <-- Creates Google Doc
|       +-- send-drive-file/         <-- Sends Drive file as email attachment
|       +-- send-email/              <-- Gmail send via OAuth
|       +-- create-calendar-event/   <-- Creates Google Calendar event
```

```
|       +-- lookup-contact/          <-- Google People API contact search
|       +-- store-google-token/      <-- Stores Google refresh token at sign-in
|       +-- get-travel-time/         <-- Google Maps Distance Matrix API
|       +-- ingest-note/             <-- Claude extraction + OpenAI embeddings
|       +-- search-knowledge/        <-- pgvector cosine similarity search
|
+-- docs/                            <-- This document
+-- schema/                          <-- Cognitive Profile TypeScript types
+-- web/                             <-- Next.js marketing page
```

5. Database Schema (Supabase Postgres)

All tables use Row-Level Security. Every insert includes `user_id` matched against `auth.uid()`.

Table	Key Columns	Purpose
google_tokens	user_id, access_token, refresh_token, expiry_date	Server-side Google OAuth token storage
calendar_events	user_id, google_event_id, summary, start_time, end_time, location	Synced Google Calendar events
gmail_messages	user_id, from_email, from_name, subject, snippet, is_important	Synced Gmail messages + sender cache for contact resolution
contacts	user_id, name, email	Contacts saved by Naavi

people	user_id, name, email, phone, relationship, notes	Rich people table for context injection
topics	user_id, subject, note, category	Preferences, concerns, profile updates
reminders	user_id, title, datetime, source	Reminders set by Naavi
knowledge_fragments	user_id, type, content, classification, confidence, embedding vector(1536)	Semantic long-term memory with pgvector
naavi_notes	user_id, title, web_view_link, created_at	Metadata for Drive docs saved via SAVE_TO_DRIVE

pgvector RPC: `search_knowledge_fragments(query_embedding, match_count, p_user_id)` performs cosine similarity search on `knowledge_fragments.embedding`.

6. Data Flow — One Full Conversation Turn

```
1. Robert taps microphone

2a. Whisper path:
    useWhisperMemo -> records MediaRecorder clip -> base64 encodes
```

```
    -> transcribe-memo Edge Function -> OpenAI Whisper API
    -> transcript text

2b. Web Speech path:
    useVoice -> SpeechRecognition API (browser-native)
    -> transcript text

3. useOrchestrator.send(transcript)

4. Person context injection (lib/memory.ts):
   extractPersonQuery() checks if Robert is asking about a person
   if yes -> getPersonContext() queries people + topics tables
   -> formatPersonContext() appends context block to message

5. naavi-client.ts -> sendToNaavi():
   -> callNaaviEdgeFunction()
   -> POST to naavi-chat Edge Function {system, messages, max_tokens:2048}
   -> Edge Function holds ANTHROPIC_API_KEY server-side
   -> calls claude-sonnet-4-6

6. Claude returns raw text

7. parseResponse() - 3-pass parser:
   Pass 1: standard JSON.parse after stripping markdown fences
   Pass 2: fix literal newlines inside string values
   Pass 3: strip control characters, fix trailing commas, quote unquoted keys
   Fallback: regex-extract speech value as last resort
   fixSentLanguage(): corrects "I sent" -> "Draft is ready" when DRAFT_MESSAGE present

8. useOrchestrator executes actions:
   SAVE_TO_DRIVE   -> saveToDrive() + saveDriveNote() to naavi_notes
   REMEMBER        -> ingestNote() -> ingest-note Edge Function
                      -> Claude extracts fragments -> OpenAI embeds -> knowledge_fragments
   CREATE_EVENT    -> create-calendar-event Edge Function -> Google Calendar
   DRIVE_SEARCH    -> search-google-drive Edge Function
   DRAFT_MESSAGE   -> accumulated in drafts[] state
   ADD_CONTACT     -> saveContact() + savePerson() -> Supabase
   SET_REMINDER    -> saveReminder() -> Supabase
   LOG_CONCERN     -> saveTopic() -> Supabase
   UPDATE_PROFILE  -> saveTopic() -> Supabase

9. speakResponse() - voice output
   Web: Web Speech Synthesis (Microsoft Aria Natural preferred)
   Native: expo-speech. Rate: 0.9

10. index.tsx re-renders with conversation bubbles + action cards
```

7. Morning Brief Pipeline

```
App mount
  -> fetchOttawaWeather() (immediate, no auth)
  -> supabase.auth.getSession() -> currentUserId
  -> Promise.all([
       fetchUpcomingEvents(7, userId),   -- calendar_events table
       fetchUpcomingBirthdays(userId),   -- calendar_events (birthday type)
       fetchImportantEmails(userId),     -- gmail_messages (is_important = true)
     ])
  -> enrichWithTravelTime(calendarItems)
       For each event with location + startISO within 8 hours:
       -> get-travel-time Edge Function
       -> Google Maps Distance Matrix API (server-side)
       -> returns { durationMinutes, distanceKm, leaveByMs, summary }
  -> setBrief() -- grouped, collapsible categories

Background sync (parallel):
  -> sync-google-calendar + sync-gmail Edge Functions
  -> re-fetch + re-enrich -> setBrief() updated silently

Leave-time alert timer (setInterval 30s):
  now >= leaveByMs AND now <= startMs + 60min:
  -> green banner "Time to leave" or red banner "Running late"
  -> tap -> opens Google Maps navigation
```

8. Action System

Action	Trigger Phrases	Execution	UI Card

`DRAFT_MESSAGE`	write / draft / compose / send / email	drafts[] state	Blue — To/Subject/Body + Send button
`ADD_CONTACT`	Name + email or phone given	saveContact() + savePerson()	Green — contact details
`SET_REMINDER`	set reminder / alert / notification	saveReminder() → Supabase	Stored — no card yet
`CREATE_EVENT`	schedule / book / meeting / appointment	create-calendar-event Edge Function	Green — tap to open in Google Calendar
`DRIVE_SEARCH`	ask about any file / document	search-google-drive Edge Function	Blue file cards with tap-to-open
`SAVE_TO_DRIVE`	save / note / store / write down / keep	save-to-drive + saveDriveNote()	Amber — tap to open in Google Docs

`REMEMBER`	remember / don't forget / keep in mind / note that	ingest-note Edge Function → knowledge_fragments	Purple — "Saved to memory — N fragments"
`LOG_CONCERN`	health / social / routine concern	saveTopic() → Supabase	Stored — no card yet
`UPDATE_PROFILE`	preference or profile change	saveTopic() → Supabase	Stored — no card yet

9. Knowledge & Memory Layer

A semantic long-term memory system combining Claude for classification and OpenAI for vector embeddings stored in pgvector.

WRITE PATH (REMEMBER ACTION)

```
Robert: "Remember that I prefer aisle seats on flights"
  -> Naavi returns REMEMBER action: { "text": "Robert prefers aisle seats on flights" }
  -> ingestNote(text, 'stated') -> ingest-note Edge Function
     1. Claude claude-sonnet-4-6 with classifier prompt
        -> extracts: [{ type, content, classification, confidence }]
        Types: life_story | important_date | preference | relationship | place | routine |
concern
        Classifications: PUBLIC | PERSONAL | SENSITIVE | MEDICAL | FINANCIAL
     2. OpenAI text-embedding-3-small (1536 dimensions) per fragment
     3. INSERT into knowledge_fragments with user_id + embedding vector
  -> UI: purple card "Saved to memory -- N fragments stored"
```

READ PATH (PERSON CONTEXT INJECTION)

```
Robert: "Tell me about my meeting with Sarah"
  -> extractPersonQuery() -> "Sarah"
  -> getPersonContext() queries people + topics tables
  -> formatPersonContext() injects into message:
     "## What Naavi knows about Sarah
      - Relationship: colleague
      - Email: sarah@example.com
      - Topics: project budget (2026-03-15)"
  -> enrichedMessage sent to Claude
```

10. Contact Resolution (3-Tier)

```
Robert: "Send an email to Heaggan"

Tier 1: SELECT name, email FROM contacts
         WHERE user_id = ? AND name ILIKE '%heaggan%'
  -> (not found)

Tier 2: SELECT from_email FROM gmail_messages
         WHERE user_id = ? AND from_name ILIKE '%heaggan%' LIMIT 1
  -> (not found)

Tier 3: Google People API -- people.searchContacts?query=heaggan
         Requires: contacts.readonly scope + People API enabled in GCP
  -> (not found)

Fallback: browser prompt() -> user enters email manually
          -> saveContact() stores for future lookups
```

11. Voice System

NAAVI

Version: 2.0 **Date:** March 19, 2026 **Status:** Phase 8 — Live **Prepared for:** CTO Review

---

	Whisper (red button, left)	Web Speech API (green button, right)
Technology	OpenAI Whisper via Edge Function	Browser SpeechRecognition API
Flow	Record clip → base64 → POST → Whisper → transcript	Browser listens → real-time transcript
Latency	~1–3s after recording stops	Near-instant
Quality	Higher accuracy, handles accents	Varies by browser/accent
Cost	OpenAI API call per use	Free (browser-native)
Availability	All browsers	Chrome / Edge / Android only

Output voice priority: Microsoft Aria Natural → Jenny Natural → Natasha Natural → Google UK English Female → Google US English → any matching language. Rate: 0.9. Language: en-CA or fr-CA.

---

12. Security Model

API KEYS — SERVER-SIDE ONLY

- `ANTHROPIC_API_KEY` — Supabase Edge Function secret
- `OPENAI_API_KEY` — Supabase Edge Function secret
- `GOOGLE_MAPS_API_KEY` — Supabase Edge Function secret
- Device holds only: Supabase anon key (public by design) + Supabase URL

AUTHENTICATION

- Google OAuth via Supabase Auth
- Google refresh token captured at SIGNED_IN event → stored in `google_tokens` table server-side
- App never holds the refresh token after sign-in

JWT / RLS

- All Edge Functions: `verify_jwt = false` in config.toml
- Auth handled inside each function via `userClient.auth.getUser()`
- All tables: RLS enabled, policies enforce `auth.uid() = user_id`

Known issue resolved: JWT key rotation (HS256 → ECC) caused 401s at the Supabase gateway. Fix: disable gateway-level JWT verification and handle auth inside each Edge Function via RLS. This is now the standard pattern for all functions.

13. Deployment Pipeline

```
Edit file locally
  -> git add [files] && git commit -m "..." && git push origin main
  -> GitHub (munk2207/naavi-app) receives push
  -> Vercel webhook: npx expo export --platform web -> dist/
```

```
  -> Live at https://naavi-app.vercel.app (~90 seconds total)

Edge Functions:
  -> npx supabase functions deploy [function-name] --no-verify-jwt
```

14. Current Capabilities (Phase 8 — Live)

CORE CONVERSATION

- Full conversation via text or voice (two input modes)
- Claude sees full conversation history + today's brief on every turn
- 3-pass JSON parser with regex fallback
- English / Canadian French bilingual

MORNING BRIEF

- Real Google Calendar events (synced)
- Real Gmail important emails (synced)
- Birthday reminders, Ottawa weather (live)
- Cards grouped by category — collapsible sections (start collapsed)
- Travel time + leave-by time on events with location
- Automatic leave-time banner (green/red) with tap-to-navigate

ACTIONS

- Draft email → one-tap send via Gmail
- Save contact → persisted to Supabase
- Create calendar event → Google Calendar
- Search Google Drive → file cards with tap-to-open
- Save note to Drive → Google Doc created
- Remember → semantic memory stored with embeddings
- Tap calendar event → opens Google Maps navigation

NOTES REPOSITORY

- Memory tab — all REMEMBER fragments, typed and classified
- Drive Notes tab — all SAVE_TO_DRIVE docs + Drive search for older files
- Pull-to-refresh

INFRASTRUCTURE

- API key fully server-side (Supabase Edge Functions)
- Google OAuth with server-side token management
- Background calendar + Gmail sync
- pgvector semantic search (1536-dimension embeddings)
- 14 Edge Functions in production

15. Known Issues & Engineering Debt

Issue	Root Cause	Status
ingest-note returned empty fragments	Claude wraps JSON in ```json fences — parser choked	Fixed: strip fences before parse
REMEMBER action not firing	System prompt rule too weak	Fixed: strengthened RULE 7 with example + more trigger phrases

Contact lookup refusal	Claude said "I don't have access to contacts"	Fixed: explicit rule forbidding this response
saveContact silently failing	Missing user_id in INSERT — RLS blocked silently	Fixed: fetch session, include user_id
JWT 401 on Edge Functions	Supabase rotated HS256 to ECC — gateway rejected tokens	Fixed: verify_jwt = false, auth via RLS
Draft card showing resolved email	UI showed resolvedEmail instead of action.to name	Fixed
Travel time not enriching after background sync	Sync overwrote brief without re-calling enrichWithTravelTime	Fixed: enrich in both load paths
People API 500	People API not enabled in GCP + JWT issue	Fixed
Old Drive notes missing	naavi_notes table is new — pre-existing docs were session-only	Partial: Drive search tab available as workaround

knowledge_fragments SELECT blocked	RLS SELECT policy missing	Fixed: added SELECT policy

16. Recommendations & Roadmap

Immediate (Next Sprint)

1. Automatic knowledge retrieval at conversation start
The `search-knowledge` Edge Function exists but is not wired into the morning brief. Recommended: at app load, run `searchKnowledge("today morning context")` and inject the top 5 relevant fragments into the system prompt. Naavi would then recall Robert's preferences automatically without being asked.

2. Confirmation cards for SET_REMINDER, LOG_CONCERN, UPDATE_PROFILE
These actions are stored but show no visual feedback. Robert has no confirmation that his reminder or concern was saved. Low-effort, high-trust improvement.

3. Drive notes backfill
A one-shot migration Edge Function should query Drive for recent Google Docs on first load (if naavi_notes is empty) and backfill the table, recovering pre-existing notes.

Medium-Term

4. Push notifications — Web Push API + Supabase pg_cron triggered at leaveByMs. Currently the leave-time banner only shows when the app is open.

5. Native mobile app (EAS Build) — Expo EAS Build produces an installable .ipa / .apk with no code changes. Gives Robert a home-screen icon and background push notifications on iOS.

6. Single voice button with automatic fallback — Two buttons cause UX confusion. Consolidate: try Web Speech API first, fall back to Whisper if unsupported or silent for 3 seconds.

---

7. Health integrations — Withings (scale + blood pressure) REST API. MyChart FHIR R4. Would make the health brief category genuinely useful.

8. Twilio SMS — Add DRAFT_SMS action type for contacts who don't use email.

Architectural

9. Conversation persistence across sessions — The history state resets on tab close. Persist conversation turns in Supabase; load last N turns at app start. Enables "as we discussed yesterday..." continuity.

10. Multi-user / caregiver mode — Extend Supabase RLS with a caregivers table and shared read policies. No data model restructuring required. A family member or home care worker could have a read-only view of Robert's brief and concerns.

17. Files a CTO Should Read First

1. **mobile/lib/naavi-client.ts** — system prompt and Claude integration. All action rules live here.
2. **mobile/hooks/useOrchestrator.ts** — conversation loop and action executor. The entire runtime flow.
3. **mobile/app/index.tsx** — the screen Robert sees. Brief pipeline, leave-time logic, action card rendering.
4. **supabase/functions/ingest-note/index.ts** — knowledge ingestion. Claude + OpenAI + pgvector composed together.
5. **mobile/lib/contacts.ts** — 3-tier contact resolution. Example of the graceful degradation pattern used throughout.
6. **supabase/functions/naavi-chat/index.ts** — API proxy. The Phase 8 security model: API key server-side, RLS auth.